The Engineering Reality of Credit Risk Data

Architecture Review: Goto Financial / Home Credit Pipeline

March 11, 2026

Executive Summary

As highlighted by the **1st Place Kaggle Winner** of the Home Credit Default Risk competition, credit underwriting is one of the most complex domains for applying machine learning. The data is fundamentally *heterogeneous*, collected over different timelines, and originates from diverse downstream systems.

While the theoretical concepts of "smart features" are well understood, this paper explores exactly how our proprietary codebase (`src/hc_risk/features.py`) implements these principles to mathematically sanitize and compress volatile banking data.

1 Data Cleansing & Safety Architecture

Before any predictive modeling can occur, the raw relational data must be made safe for consumption by gradient boosting algorithms. Left unchecked, silent data anomalies in finance completely invert risk profiles.

1.1 1. Sentinel Value Decoupling

In legacy systems, missing data is rarely recorded as `NULL`. Frequently, programmers inject "sentinel values" (impossible extreme numbers) as placeholders. Our codebase handles the most notorious of these—the employment tenure sentinel:

```
# Handling the 'Days Employed' sentinel
frame["DAYS_EMPLOYED_ANOM"] = (frame["DAYS_EMPLOYED"] == 365243).astype(int)
frame.loc[frame["DAYS_EMPLOYED"] == 365243, "DAYS_EMPLOYED"] = np.nan
```

A value of 365,243 equates to 1,000 years of employment. If passed blindly into a machine learning model, the algorithm concludes that these applicants possess infinite job stability, drastically mispricing their risk. By introducing an explicit `DAYS_EMPLOYED_ANOM` binary flag and nullifying the raw value, the model correctly isolates the "unemployed/retired" segment as a distinct risk cohort.

1.2 2. Ratio Engineering Constraints

The backbone of the *Capacity Pillar* involves dividing debt by income. In a large-scale database, dividing by zero occurs almost constantly (e.g., applicants with $0 reported debt). A single `ZeroDivisionError` will cascade and fail an automated pipeline.

Our explicit constraint wrapper resolves this:

```python
def safe_divide(numerator: pd.Series, denominator: pd.Series):
    if isinstance(denominator, pd.Series):
        denominator = denominator.replace({0: np.nan})
    return numerator / denominator
```

This transforms a fatal programming error into a `NaN` (Not a Number) value. Tree-based learners natively route `NaN` values down designated split paths, converting a mathematical exception into a valid predictive signal (i.e., "the borrower has no history of this denominator").

1.3 3. Memory Optimization for Flattened Matrices

The process of flattening 1-to-N transaction logs into single applicant profiles creates massively wide data matrices. Our feature generation pipeline outputs upwards of 700 to 1,000 distinct columns. To prevent out-of-memory (OOM) crashes in production, the codebase aggressively downcasts 64-bit precision formats immediately upon ingestion:

```python
def reduce_memory(df: pd.DataFrame) -> pd.DataFrame:
    for column in df.columns:
        if pd.api.types.is_float_dtype(df[column]):
            df[column] = pd.to_numeric(df[column], downcast="float")
        elif pd.api.types.is_integer_dtype(df[column]):
            df[column] = pd.to_numeric(df[column], downcast="integer")
    return df
```

This deterministic downcasting shrinks the dataframe's RAM footprint by 40-50%, enabling in-memory joins during distributed model training.

2 The "Many-to-One" Aggregation Pipeline

The winner explicitly notes: *"These features employed various forms of aggregation on many-to-one tables, and joined them with application features."*

Our codebase implements this via distinct temporal aggregation functions designed to compress transaction histories into behavioral summaries.

2.1 1. Synthesizing Missing Business Logic (Interest Rates)

The original relational database does not provide the interest rate for previous loans—a massive missing signal for assessing risk appetite. The codebase manually synthesizes a proxy for this by reverse-engineering the monthly payment logs in `previous_application_aggregate()`:

```python
previous["TOTAL_REPAYMENT_RATIO"] = safe_divide(
    previous["AMT_ANNUITY"] * previous["CNT_PAYMENT"],
    previous["AMT_CREDIT"],
)
previous["ESTIMATED_INTEREST_RATE"] = previous["TOTAL_REPAYMENT_RATIO"] - 1.0
```

High inferred interest rates mathematically target subprime, high-risk prior borrowing behavior that was otherwise invisible in the raw tables.

2.2 2. Time-Decay Targeting (Recency Weighting)

Historical data loses predictive power as it ages. Taking a simple average of someone's past late payments penalizes a mistake made 5 years ago exactly as much as a mistake made yesterday. The codebase explicitly prevents this by engineering a Recency Weight in `installments_aggregate()`:

```python
installments["recency_weight"] = 1.0 / (installments["DAYS_INSTALMENT"].abs() +
    1.0)
```

A late payment from 1 month ago mathematically exerts substantially more influence on the final borrower profile than a late payment from 40 months ago.

2.3 3. Distinguishing "Lazy" vs "Distressed" Borrowers

In the installment logs, simply checking if a borrower paid late is too blunt a metric. The codebase engineers threshold flags in `installments_aggregate()` to classify the severity of the deterioration:

```
installments["DAYS_PAST_DUE"] = (
    installments["DAYS_ENTRY_PAYMENT"] - installments["DAYS_INSTALMENT"]
).clip(lower=0)
installments["late_payment_flag"] = (installments["DAYS_PAST_DUE"] > 0).astype(
    "int8")
installments["severe_late_flag"] = (installments["DAYS_PAST_DUE"] >= 30).astype
    ("int8")
```

A `late_payment_flag` (1 day late) often indicates banking friction or forgetfulness. A `severe_late_flag` (30+ days late) indicates genuine financial distress. The model captures both as distinct predictive vectors.

3 Conclusion

While modern algorithms (`LightGBM`, `XGBoost`) provide the statistical heavy lifting, they are entirely dependent on the structural integrity of the input data. Our codebase demonstrates that applying machine learning to credit underwriting is less about algorithmic tuning and vastly more dependent on robust, defensively engineered data pipelines that mathematically capture human financial behavior.